

Principles of Distributed Systems

第10周：分布式基于对象和分布式文件系统

Week 10: Distributed Object-Based and Distributed File Systems

By: Fahad Sabah

计算机学院,

北京工业大学,中国

College of Computer Science,
Beijing University of Technology, China

fahad.sabah@bjut.edu.cn



分布式基于对象的系统

Distributed Object-Based Systems

- ❑ **Distributed objects:** proxy, skeleton, interface/state
- ❑ **Object servers:** adapters and activation policies
- ❑ **Communication:** binding, RMI, parameters, async calls
- ❑ Naming, synchronization, replication, fault tolerance, security
- ❑ Problems, algorithms, small programs
- ❑ 分布式对象：代理、骨架、接口/状态
- ❑ 对象服务器：适配器与激活策略
- ❑ 通信：绑定、RMI、参数、异步调用
- ❑ 命名、同步、复制、容错、安全性
- ❑ 问题、算法、小程序

为什么选择分布式对象？

Why distributed objects?

- ❑ Everything is treated as an object: resources and services are offered through object interfaces.
- ❑ Clients invoke methods instead of directly manipulating remote data.
- ❑ The interface hides object location, communication, marshaling, and server details.
- ❑ Useful because distributed-system complexity is hidden behind a familiar programming model.

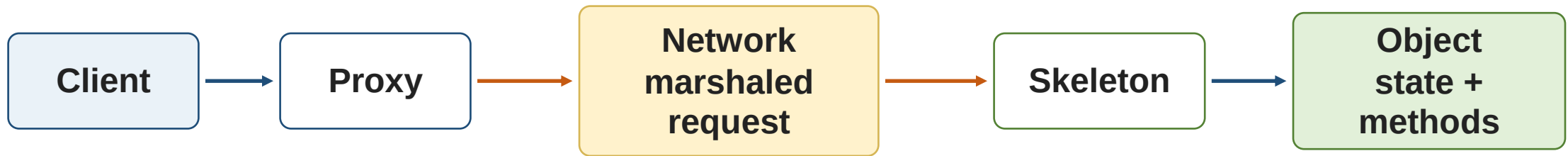
Local style
`calculator.add(2,3)`

Remote style
`remoteCalculator.add(2,3)`

Quiz: Why is hiding object location useful?

基本体系结构

Basic distributed-object architecture



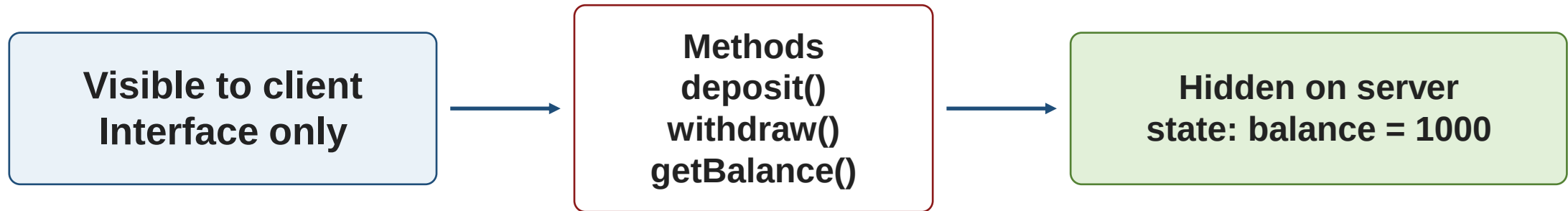
Proxy: marshal call, send message,
unmarshal reply

Skeleton: unmarshal request, invoke
method, marshal reply

```
client calls: obj.add(2, 3)
actual execution: serverObject.add(2, 3)
```

接口与状态

Interface vs state



```
class BankAccount:
    def deposit(self, amount):
        self.balance += amount

# Avoid direct state access:
# account.balance = 999999
```

Quiz
Why is direct access to object state dangerous?

对象类型

Types of objects

Type	Meaning	Example
Remote object	State normally resides on one server	Java RMI calculator
Distributed shared object	State may be replicated/distributed	Globe object
Transient object	Exists only while server/session exists	Temporary calculator
Persistent object	Survives server shutdown	Customer record in database

Memory problem
100 transient objects × 2 MB = 200 MB

编译时对象与运行时对象

Compile-time vs runtime objects

Compile-time object

- Class known before execution
- Common in Java/C++
- Easy for programmers

Runtime object

- Created/wrapped during execution
- Language-independent
- Object adapter can wrap old code

```
class Calculator {  
    int add(int a, int b) { return a + b; }  
}
```

Quiz: Which approach is better for multi-language systems?

示例：企业级Java Beans

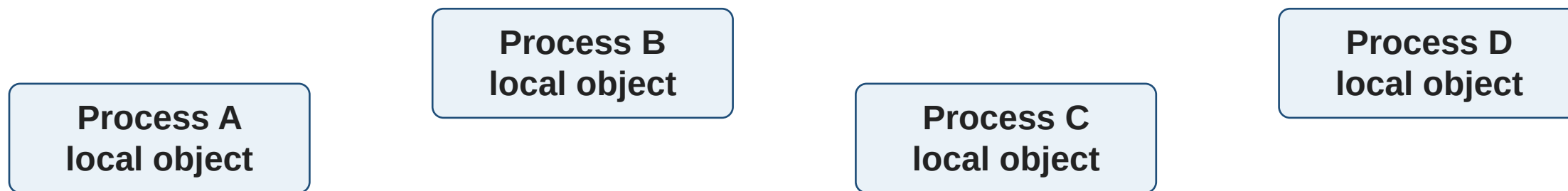
Example: Enterprise Java Beans

Bean type	Meaning	Simple example
Stateless session bean	No client state	Currency converter
Stateful session bean	Keeps session state	Shopping cart
Entity bean	Long-lived persistent object	Customer profile
Message-driven bean	Reacts to messages	Order notification handler

Quiz: A shopping cart remembers selected items. Which bean type?
Answer: Stateful session bean

Globe分布式共享对象

Globe distributed shared objects



One distributed shared object spans multiple processes

Subobject	Role
Semantics	Actual functionality
Communication	Network interface
Replication	Distribution/replication policy
Control	Connects user interface to replication

对象服务器和适配器

Object Servers and Adapters

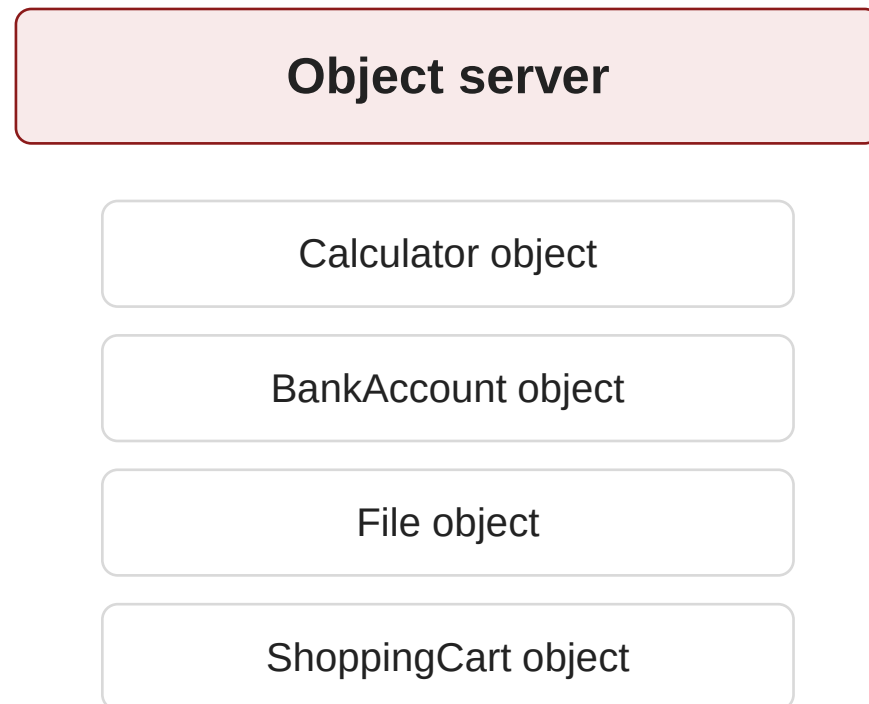
Servers host objects; adapters manage activation policies.

对象服务器

Object server

- ❑ 对象服务器是分布式对象居住的地方。
- ❑ 服务器本身并不提供固定服务。
- ❑ 服务由服务器内部的对象实现。
- ❑ 更改服务 = 添加、删除或更新对象。

- ❑ An object server is a place where distributed objects live.
- ❑ The server itself does not offer one fixed service.
- ❑ Services are implemented by objects inside the server.
- ❑ Changing service = add, remove, or update objects.



激活政策

Activation policies

Create at server startup

Advantage: fast first call

Disadvantage: wastes memory

Create on first request

Advantage: saves memory

Disadvantage: first call is slower

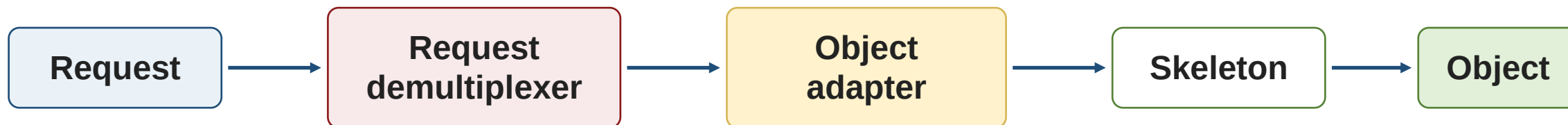
Numerical problem

1000 possible objects × 5 MB = 5000 MB

100 used objects × 5 MB = 500 MB

Memory saved = 4500 MB

对象适配器 Object adapter



Algorithm: object adapter request handling

1. Receive request and extract object reference.
2. Find correct adapter and activation policy.
3. If target object is inactive, activate it.
4. Forward request to skeleton.
5. Skeleton invokes method and returns result.

代理/骨架程序

Proxy/skeleton program

```
class Calculator:
    def add(self, a, b):
        return a + b

class Skeleton:
    def __init__(self, obj): self.obj = obj
    def handle_request(self, method, args):
        return getattr(self.obj, method)(*args)

class Proxy:
    def __init__(self, skeleton): self.skeleton = skeleton
    def add(self, a, b):
        return self.skeleton.handle_request("add", (a, b))

proxy = Proxy(Skeleton(Calculator()))
print(proxy.add(10, 20))    # 30
```

**The client calls proxy.add().
The actual method runs in
Calculator on the server side.**

Quiz
**Which object marshals the client
method call?**

沟通

Communication

Binding, object references, RMI, parameter passing, and asynchronous calls.

绑定客户端到对象

Binding a client to an object

- ❑ 客户端在调用远程对象之前，必须绑定该对象。
 - ❑ 绑定通常会创建或获得本地代理。
 - ❑ 隐式绑定：当使用引用时自动执行。
 - ❑ 显式绑定：客户端先调用 `bind (reference)` 。
-
- ❑ Before a client invokes a remote object, it must bind to the object.
 - ❑ Binding usually creates or obtains a local proxy.
 - ❑ Implicit binding: done automatically when reference is used.
 - ❑ Explicit binding: client calls `bind(reference)` first.

Explicit binding:

```
obj_ref = ...  
proxy = bind(obj_ref)  
proxy.do_something()
```

Implicit binding:

```
obj_ref.do_something()
```

Quiz: What does binding usually return?
Answer: a proxy

对象引用

Object references

A reference must contain enough information to find and invoke the object.

Host address
192.168.1.10

Port
5000

Object ID
calculator_01

Protocol
TCP/IIOP

Numerical problem

Host 4 bytes + Port 2 bytes + Object ID 16 bytes + Protocol ID 2 bytes = 24 bytes

静态与动态远程调用

Static vs dynamic remote method invocation

Static invocation
Method known at compile time

`calculator.add(2, 3)`

Dynamic invocation
Method selected at runtime

`invoke(calculator, "add", [2,3])`

```
class Calculator:
    def add(self,a,b): return a+b
    def multiply(self,a,b): return a*b

def invoke(obj, method_name, params):
    return getattr(obj, method_name)(*params)

print(invoke(Calculator(), "add", [2,3]))      # 5
print(invoke(Calculator(), "multiply", [2,3])) # 6
```

参数传递

Parameter passing

Parameter type	Common behavior	Cost
Local object	Passed by value: copy whole object	Can be expensive
Remote object	Passed by reference: copy reference only	Usually small
Primitive value	Passed by value	Usually cheap

Numerical problem

Local object: $10 \text{ MB} \times 5 \text{ servers} = 50 \text{ MB}$

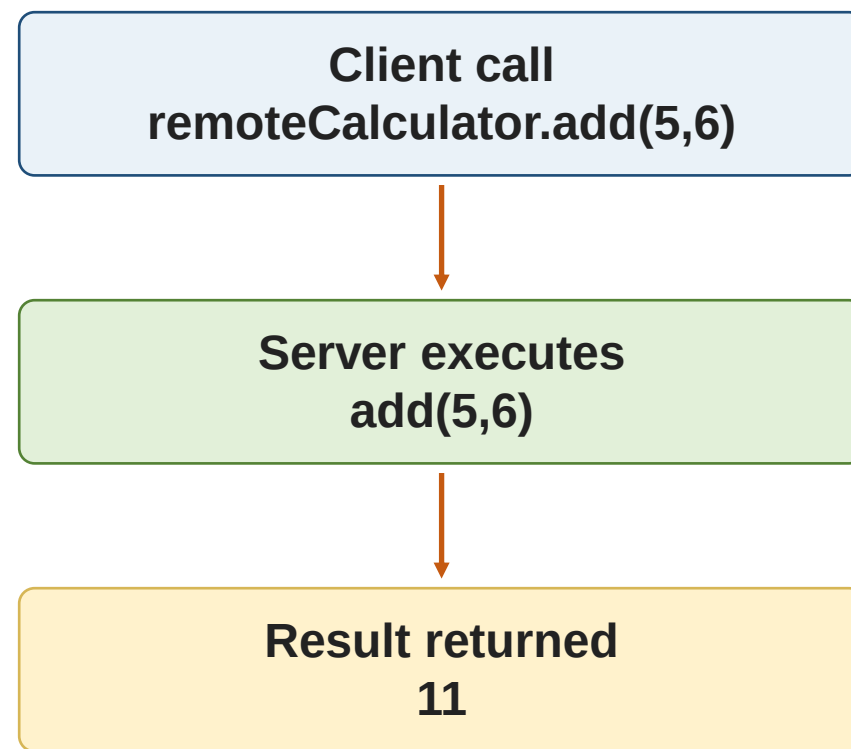
Remote reference: $200 \text{ bytes} \times 5 = 1000 \text{ bytes}$

Discussion: Why is passing large local objects by value expensive?

Java RMI概念

Java RMI concept

- ❑ Java RMI 允许一台机器上的 Java 对象从另一台机器调用。
 - ❑ 远程对象状态保持在服务器上。
 - ❑ 客户端使用代理/存根。
 - ❑ 可序列化的本地对象可以作为参数复制。
 - ❑ 远程对象通过引用传递。
-
- ❑ Java RMI lets a Java object on one machine be invoked from another machine.
 - ❑ Remote object state stays on the server.
 - ❑ The client uses a proxy.
 - ❑ Serializable local objects can be copied as parameters.
 - ❑ Remote objects are passed by reference.



同步与异步调用

Synchronous vs asynchronous calls

Synchronous
Client waits for result

Asynchronous
Client continues working; result arrives later

```
import time, threading

def remote_add(a, b):
    time.sleep(2)
    print("Result:", a+b)

threading.Thread(target=remote_add, args=(10,20)).start()
print("Client continues working...")
```

Quiz: Which call style is better when the operation is slow?

回调模型

Callback model

- ❑ Client sends request and provides a callback function/object.
- ❑ Runtime or server calls back when result is ready.
- ❑ Useful when the client should not repeatedly check for results.

Key idea: result is pushed back to the client.

```
import threading, time

def async_add(a, b, callback):
    def task():
        time.sleep(2)
        callback(a + b)

    threading.Thread(target=task).start()

def my_callback(result):
    print("Callback result:", result)

async_add(4, 5, my_callback)
print("Client is free.")
```

轮询模型

Polling model

- ❑ Client sends request.
- ❑ Client repeatedly asks: “Is the result ready?”
- ❑ Easy to implement, but frequent polling can waste CPU/network resources.

Quiz: Which model may waste CPU if checking too often?
Answer: Polling

```
result = None

def remote_add(a, b):
    global result
    time.sleep(2)
    result = a + b

threading.Thread(target=remote_add,
args=(7,8)).start()
while result is None:
    print("Not ready. Polling...")
    time.sleep(0.5)
print("Result:", result)
```

命名、同步、复制

Naming, Synchronization, Replication

Finding objects and keeping them correct under concurrency and replication.

命名与对象引用

Naming and object references

System	Reference style	How object is found
CORBA	IOR: Interoperable Object Reference	Reference can include host, port, object key
Globe	OID: Object Identifier	Lookup OID in location service first

**CORBA style: direct
reference**

**Globe style: indirect
reference**

CORBA IOR组成

CORBA IOR components

Repository ID

Profile ID

Host

Port

Object key

Components

Example

Interface: Calculator | Host: 192.168.1.20 | Port: 9000 | Object key: calc001 | Protocol: IIOP

Quiz: Which fields locate the server?
Answer: host and port

Globe绑定算法

Globe binding algorithm

1. Client has an object identifier (OID).
2. Client asks the location service for contact address.
3. Location service returns address information.
4. Client loads or creates local representative.
5. Client invokes methods through the local representative.

OID alone is not enough to contact the object.

OID → Location Service → Contact Address → Local Representative

同步问题

Synchronization problem

- ❑ Multiple clients may invoke the same object at the same time.
- ❑ Without synchronization, object state can become inconsistent.
- ❑ Remote locking is harder because clients may crash or be on different machines.

Initial balance = 100

Client A: deposit +50

Client B: withdraw -30

Correct final balance = 120

锁保护对象状态

Locking object state

```
import threading

class BankAccount:
    def __init__(self):
        self.balance = 100
        self.lock = threading.Lock()

    def deposit(self, amount):
        with self.lock:
            self.balance += amount

    def withdraw(self, amount):
        with self.lock:
            self.balance -= amount
```

Algorithm

1. Receive method request.
2. Acquire object lock.
3. Execute method.
4. Update object state.
5. Release object lock.
6. Return result.

locks serialize access to internal state.

复制对象一致性

Replication consistency

- ❑ Replication improves availability, fault tolerance, and performance.
- ❑ But all replicas must see updates in the same order.
- ❑ If operations are not ordered, replicas may diverge.

Replica 1 balance = 100

Replica 2 balance = 100

Replica 3 balance = 100

After deposit +50, all should become 150

全序调用

Totally ordered invocation

Initial $x = 10$

M1: $x = x + 5$

M2: $x = x \times 3$

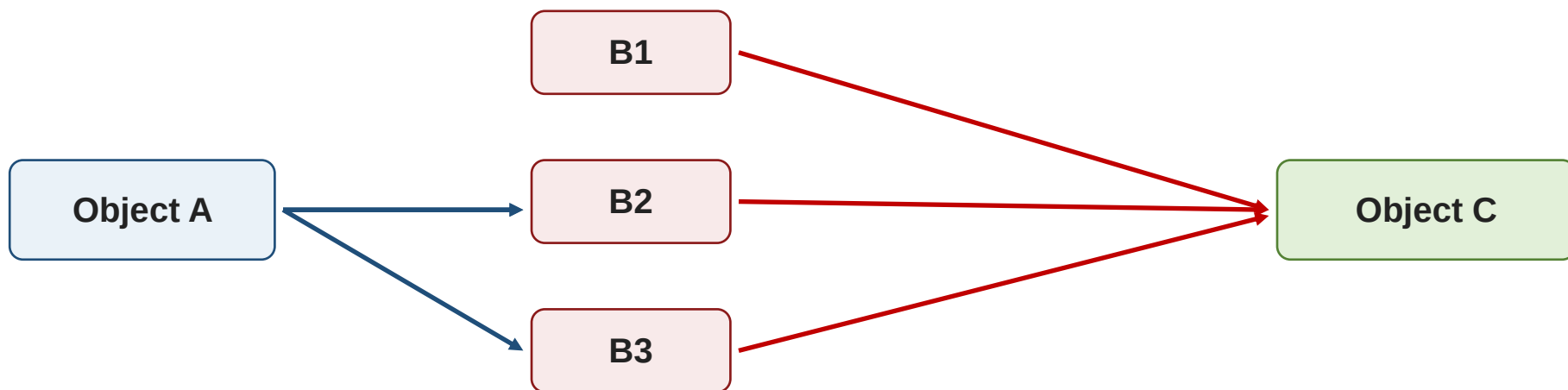
Replica A: M1 then M2
 $(10 + 5) \times 3 = 45$

Replica B: M2 then M1
 $(10 \times 3) + 5 = 35$

Conclusion: different ordering gives inconsistent replicas.

复制调用问题

Replicated invocation problem



Problem: if B is replicated, C may receive the same call 3 times.

Danger: transfer(\$100000) may be executed multiple times.

避免重复调用算法

Algorithm: avoiding duplicate replicated calls

1. Replicated object B receives request.
2. All replicas assign the same request ID.
3. Select one coordinator replica.
4. Coordinator forwards request to object C.
5. Other replicas suppress duplicate request.
6. C executes request once.
7. Coordinator returns result to replicas of B.

Coordinator principle
Only one replica forwards the outgoing call.

Quiz
Why should only one replica forward the request?

Fault Tolerance and Security

Object groups hide failures; authentication and authorization protect invocations.

容错与对象组

Fault tolerance and object groups

- ❑ Fault-tolerant systems replicate objects into object groups.
- ❑ A client sees the group as one logical object.
- ❑ If one replica fails, another replica can continue service.

Strategy	Idea
Primary-backup	Primary handles requests; backups take over after failure
Active replication	All replicas execute the same requests
Quorum replication	Operation needs agreement from enough replicas

Numerical problem
5 replicas, majority quorum = $\text{floor}(5/2)+1 = 3$
If 2 fail, remaining 3 → system can continue.

容错CORBA

Fault-tolerant CORBA

- ❑ CORBA可以将对象复制到对象组中。
 - ❑ 对象组可以被引用为单个对象。
 - ❑ IOGR = 互操作对象组参考。
 - ❑ 如果绑定到一个副本失败，运行时可以尝试另一个副本。
-
- ❑ CORBA can replicate objects into object groups.
 - ❑ An object group can be referenced as if it were a single object.
 - ❑ IOGR = Interoperable Object Group Reference.
 - ❑ If binding to one replica fails, the runtime can try another replica.

Normal IOR
one object

IOGR
group of replicas

Quiz: Why is an object group useful?
Answer: hides failures and improves availability.

容错Java困难点

Why fault-tolerant Java is difficult

- ❑ Active replication requires deterministic behavior.
- ❑ The JVM may behave nondeterministically.
- ❑ Thread scheduling can differ across replicas.
- ❑ Native code, I/O, and shared variables may produce different results.

Replica JVM 1
Thread order: T1 → T2

Replica JVM 2
Thread order: T2 → T1

Different schedule may produce different state.

分布式对象安全

Security in distributed objects

Question	Security concept
Is this the correct object/server?	Authentication
Is this client allowed to call this method?	Authorization
Can attackers read/modify the request?	Encryption + signature
Can the downloaded proxy be trusted?	Signed code / certificate

Key mechanisms: authentication, authorization, encryption, signatures, certificates

安全方法调用

Secure method invocation

1. Client requests method.
2. Client identity is checked.
3. Client permission is checked.
4. Secure channel is created.
5. Request is encrypted and signed.
6. Server verifies request.
7. Server checks authorization.
8. Method is executed.
9. Result is returned securely.

Authentication = Who are you?

Authorization = What can you do?

Encryption = Keep data secret

权限位串问题

Numerical problem: access-control bit string

Methods: M0 M1 M2 M3 M4 M5 M6 M7

Permission bit string: 00101100

Assume left-to-right maps to M0...M7.
Bits with value 1 are positions 2, 4, and 5.
Allowed methods: M2, M4, M5.

权限检查程序

Simple permission-check program

```
methods =  
["M0", "M1", "M2", "M3", "M4", "M5", "M6", "M7"]  
permission = "00101100"  
  
allowed = []  
for i, bit in enumerate(permission):  
    if bit == "1":  
        allowed.append(methods[i])  
  
print("Allowed methods:", allowed)  
# Output: ['M2', 'M4', 'M5']
```

Student task
Modify permission to
10101000.
Which methods are allowed?

Answer
M0, M2, M4

建议的班级活动

Review Problems and Quizzes

数值练习1：RMI延迟

Numerical exercise 1: RMI latency

Remote method call costs:

Step	Time
Marshaling	2 ms
Network request	10 ms
Server execution	5 ms
Network reply	10 ms
Unmarshaling	2 ms

Total time = 2 + 10 + 5 + 10 + 2 = 29 ms

数值练习2：传值与传引用

Numerical exercise 2: value vs reference

Local object size = 4 MB
Sent to 6 servers
Data sent = $4 \times 6 = 24$ MB

Remote object reference = 300 bytes
Sent to 6 servers
Data sent = $300 \times 6 = 1800$ bytes

Conclusion: passing references is much cheaper for large objects.

数值练习3：复制顺序

Numerical exercise 3: replica ordering

Initial value: $x = 20$

A: $x = x - 5$
B: $x = x \times 2$

Replica 1: A then B
 $(20 - 5) \times 2 = 30$

Replica 2: B then A
 $(20 \times 2) - 5 = 35$

Replicas are inconsistent.

数值练习4：多数派仲裁

Numerical exercise 4: majority quorum

A replicated object has 7 replicas.
Majority quorum is required.
How many replicas must agree?

$$\text{majority} = \text{floor}(7/2) + 1 = 4$$

Answer: 4 replicas

选择题小测

Multiple-choice quiz

Question	Options	Answer
What does a proxy do?	A Store state B Convert method calls into network messages	B
What is a skeleton?	A Client data copy B Server-side unmarshaling/invoke component	B
In Java RMI, local objects are passed by...	A Reference B Value	B
A remote object is usually passed by...	A Value B Reference	B

判断题小测

True/False quiz

Statement	Answer
A distributed object hides its internal state behind an interface.	True
A transient object survives after the server shuts down.	False
Object references may contain host, port, and object ID.	True
Replication always makes consistency easier.	False
Authentication checks whether a user is allowed to call a method.	False

课堂总结

Summary

- ❑ Distributed objects allow clients to invoke methods on remote objects.
- ❑ Core components: proxy, skeleton, object server, object adapter, object reference.
- ❑ Communication issues: binding, static/dynamic RMI, parameter passing, async calls.
- ❑ Correctness issues: synchronization, consistency, replication, fault tolerance.
- ❑ Security issues: authentication, authorization, secure channel, trusted proxies.

Take-home message

Distributed objects make remote resources look like local objects, but correctness still depends on careful communication, synchronization, replication, and security.

分布式文件系统

Distributed File Systems

Definition

A distributed file system lets users and programs access files stored on remote machines as if they were local files.

User A computer → reads file from remote file server
User B computer → writes file to same remote file server

- ❑ The system should hide location details.
- ❑ It should support normal file operations.
- ❑ It should be reliable, secure, and efficient.

Goal: remote files should feel local

什么是分布式文件系统?

What Is a Distributed File System?

- ❑ Stores files on one or more remote machines.
- ❑ Users access files with familiar operations.
- ❑ The client should not need to know the exact physical location.

Basic operations:

open	close
read	write
create	delete
rename	

Local: `open("notes.txt")`

Distributed: `open("/remote/server/notes.txt")`

为什么我们需要分布式文件系统?

Why Do We Need Distributed File Systems?

File sharing

Many users access the same data.

Remote access

Work from different machines.

Collaboration

Teams can share project files.

Scalability

Store large data across servers.

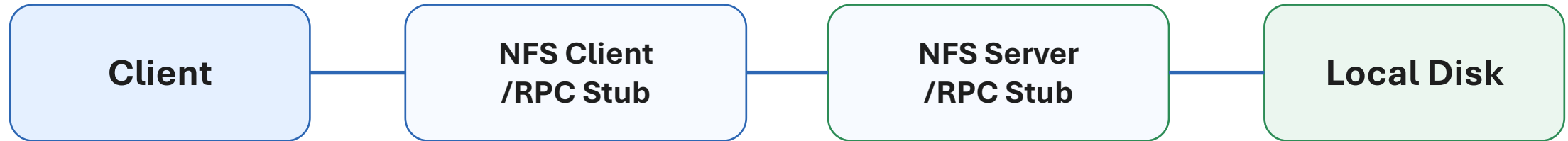
Fault tolerance

Replicas help survive failures.

Example: students in a lab access the same course files from different computers.

基本客户端-服务器架构

Basic Client-Server Architecture



- ❑ The client sends read/write requests.
- ❑ The server performs the actual file operation.
- ❑ NFS is the classic example of this model.

远程访问模式与上传/下载模式

Remote Access Model vs Upload/Download Model

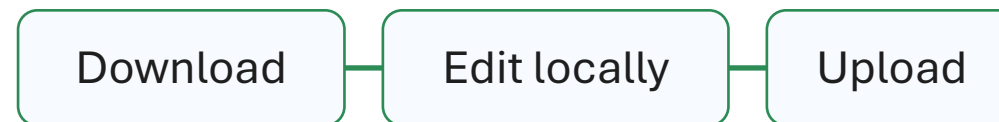
Remote access model

File stays on the server. Client sends operations such as READ and WRITE.



Upload/download model

Client downloads the whole file, edits it locally, and uploads it back.



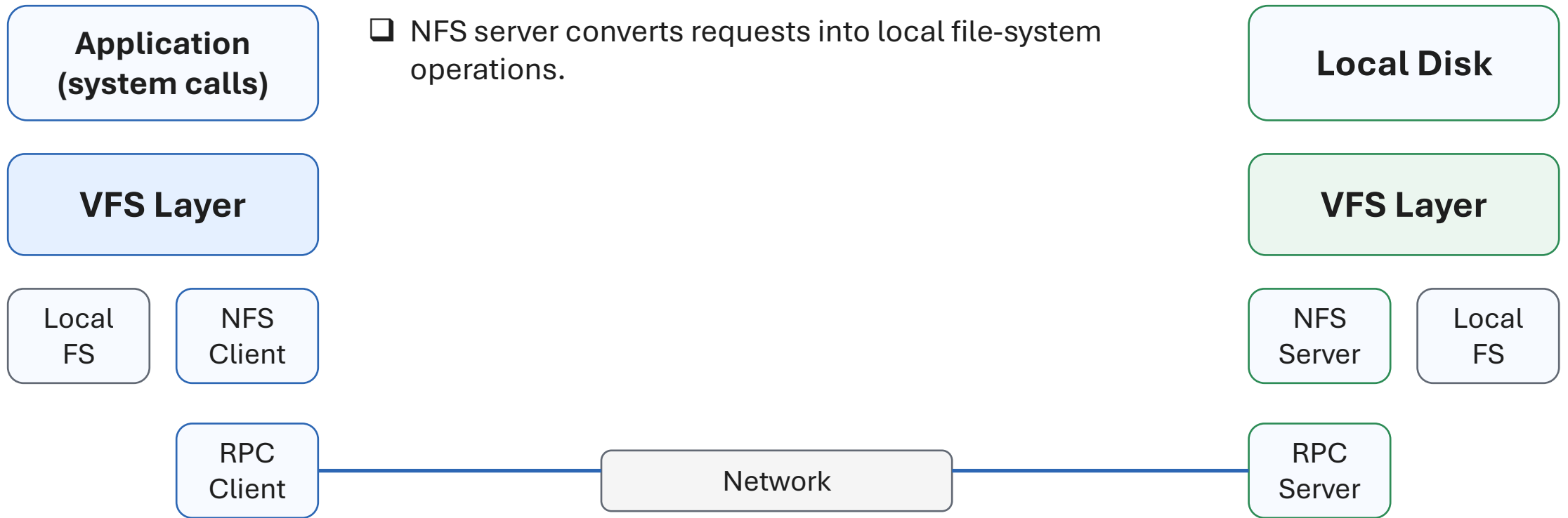
Remote access: good for partial access, but many requests can be slow.

Upload/download: good for local editing, but conflicts may occur.

NFS架构简明扼要

NFS Architecture in Simple Terms

- ❑ VFS hides differences between file systems.
- ❑ NFS client turns file operations into RPC requests.
- ❑ NFS server converts requests into local file-system operations.



NFS文件操作

NFS File Operations

1. Create, remove, rename files.
2. Look up names and read directory entries.
3. Get and set file attributes.
4. Read and write file data.
5. NFSv4 adds open and close operations.

Common operations:

LOOKUP	READ
WRITE	GETATTR
SETATTR	RENAME
OPEN	CLOSE

基于集群的分布式文件系统

Cluster-Based Distributed File Systems

Whole-file distribution

Different complete files are stored on different servers.

Server 1
File A

Server 2
File B

Server 3
File C

File striping

One large file is split into parts and distributed across servers for parallel access.

Server 1
Part 1

Server 2
Part 2

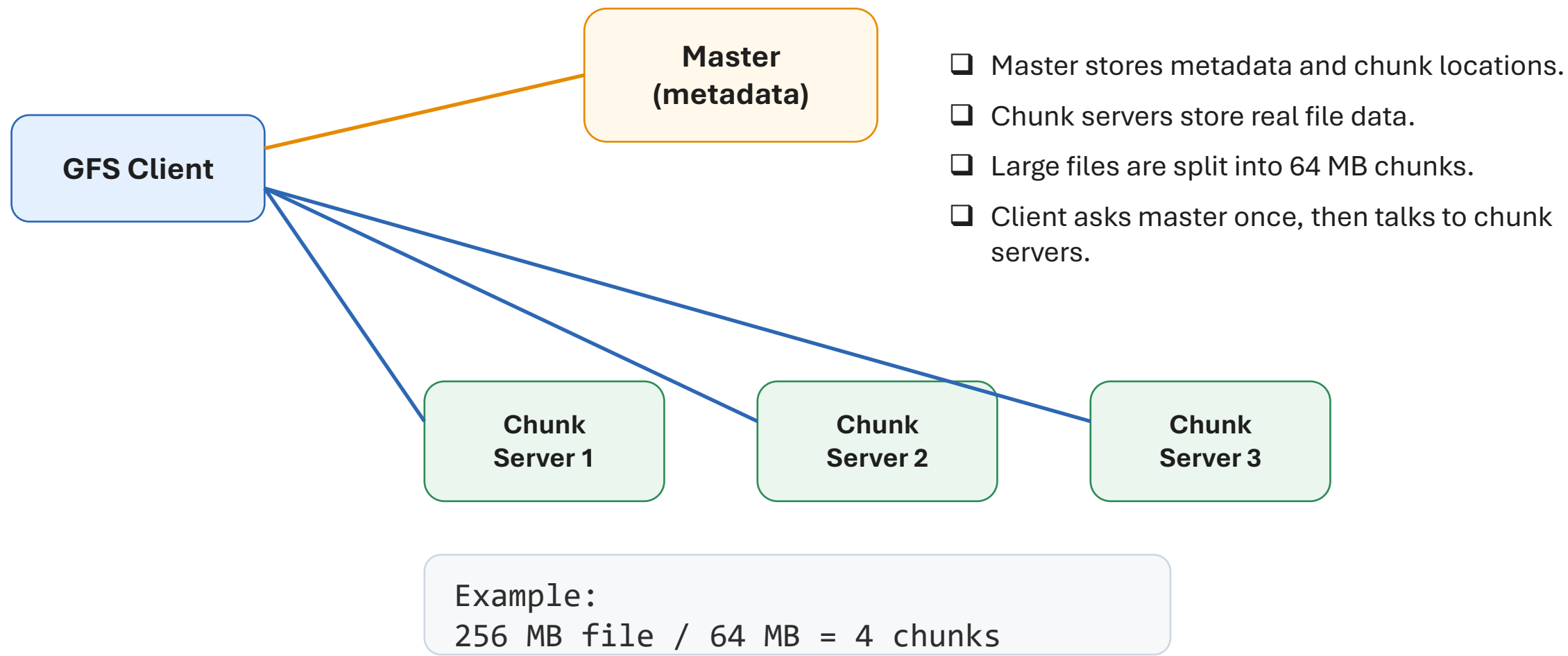
Server 3
Part 3

Numerical example:

$300 \text{ MB file} / 3 \text{ servers} = 100 \text{ MB per server}$

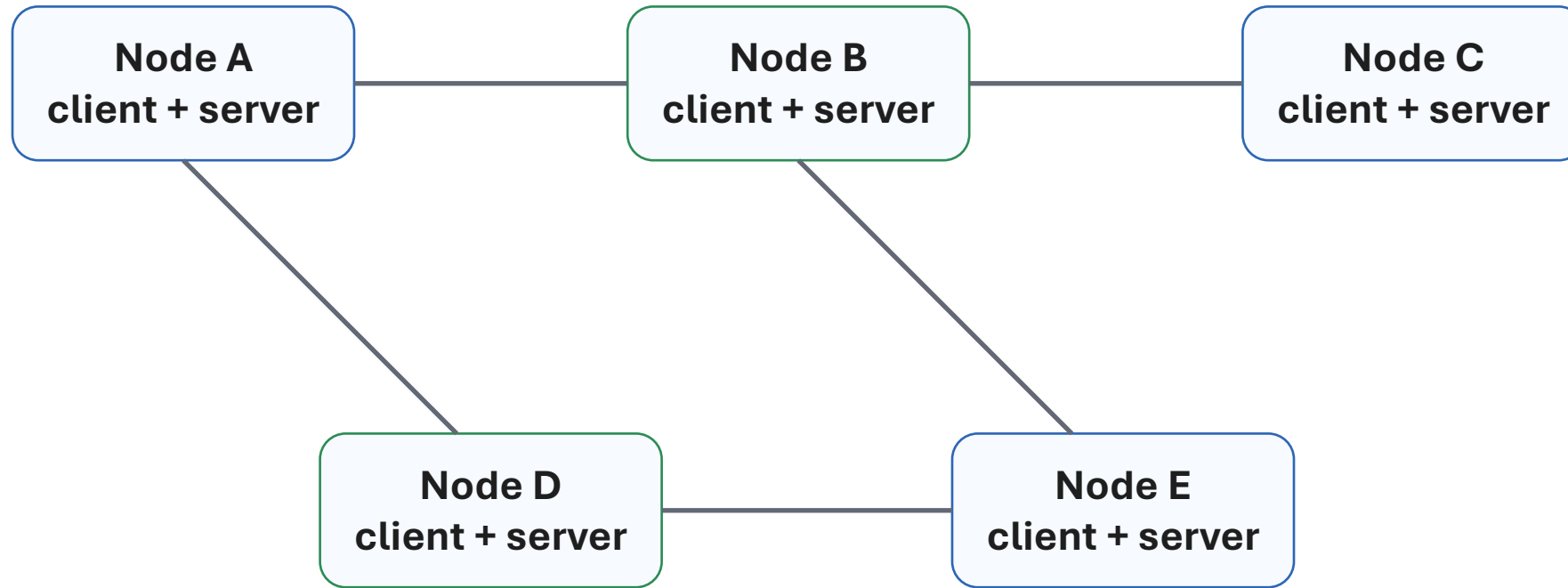
谷歌文件系统：简易查看

Google File System: Easy View



点对点文件系统

Peer-to-Peer File Systems



- ☐ No single central server is required.
- ☐ Each node may store some files or blocks.
- ☐ A lookup system helps find where data is stored.
- ☐ Examples use DHTs such as Chord.

无状态服务器与有状态服务器

Stateless vs Stateful Servers

Stateless server

Does not remember client information between requests. Recovery is simpler.

Stateless:

READ block 1

READ block 2

Each request stands alone

Stateful server

Remembers opened files, locks, delegations, and client cache information.

Stateful:

OPEN file

READ / WRITE

CLOSE file

Server remembers session state

NFSv3 vs NFSv4

Feature	NFSv3	NFSv4
Server style	Mostly stateless	More stateful
Open/close	No regular open/close	Supports open/close
Compound ops	No	Yes
WAN support	Limited	Better
Callbacks	No	Yes

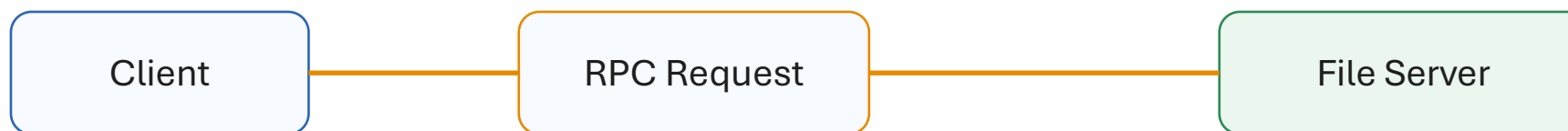
Key message: NFSv4 added stateful features to improve performance and wide-area use.

NFS中的RPC通信

RPC Communication in NFS

- ❑ Distributed file systems often use Remote Procedure Calls.
- ❑ Client calls a remote file operation like a local function.
- ❑ RPC hides message sending and marshaling details.

```
read(file_handle, offset, size)  
write(file_handle, offset, data)  
getattr(file_handle)
```



NFSv3 读取与 NFSv4 复合请求

NFSv3 Read vs NFSv4 Compound Request

NFSv3

1. LOOKUP file name
 2. READ file data
- Two RPC round trips may be needed.

NFSv4

LOOKUP + OPEN + READ can be grouped into one compound request.

If one RPC round trip = 20 ms

NFSv3: $2 \times 20 \text{ ms} = 40 \text{ ms}$

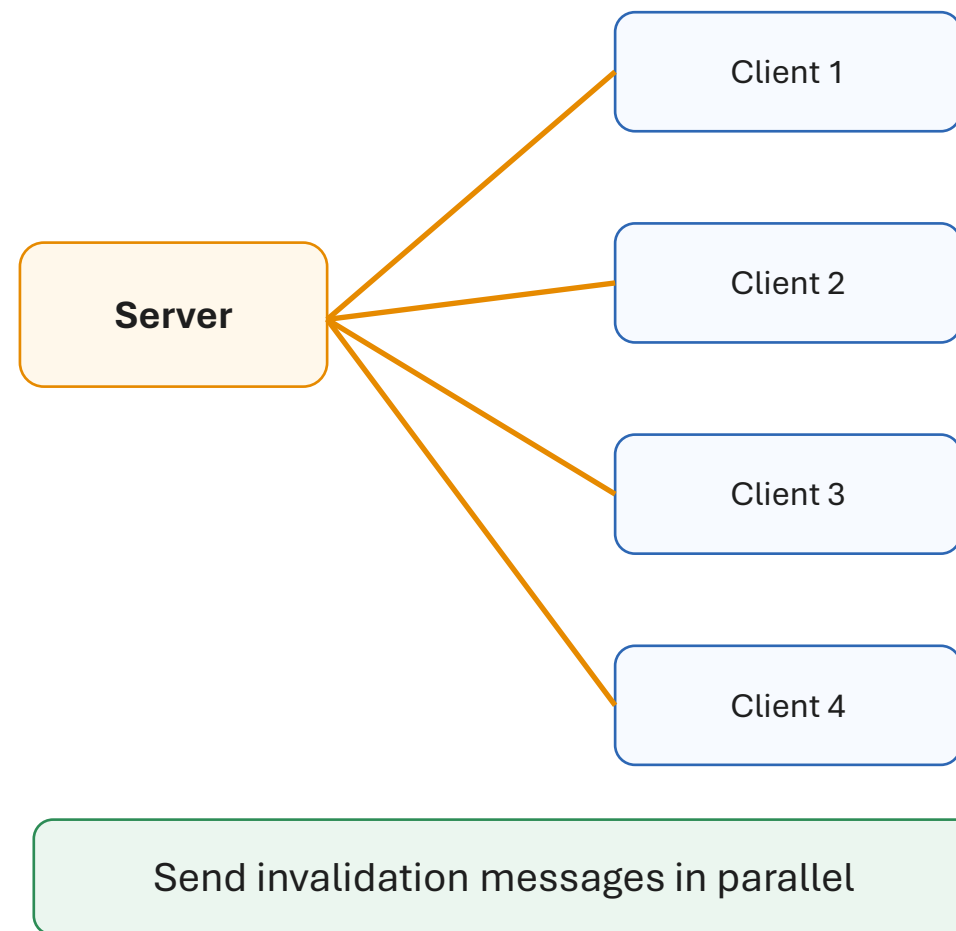
NFSv4: $1 \times 20 \text{ ms} = 20 \text{ ms}$

Saved: 20 ms

缓存失效的多播RPC

Multicast RPC for Cache Invalidation

- ❑ A server may need to notify many clients.
- ❑ Example: cached file copies become invalid.
- ❑ Sequential invalidation can be slow if one client fails.
- ❑ Parallel invalidation is faster.



命名与安装

Naming and Mounting

- ❑ Files are usually organized in a hierarchy.
- ❑ Mounting attaches a remote file system into the local name space.
- ❑ After mounting, remote files appear under a local path.

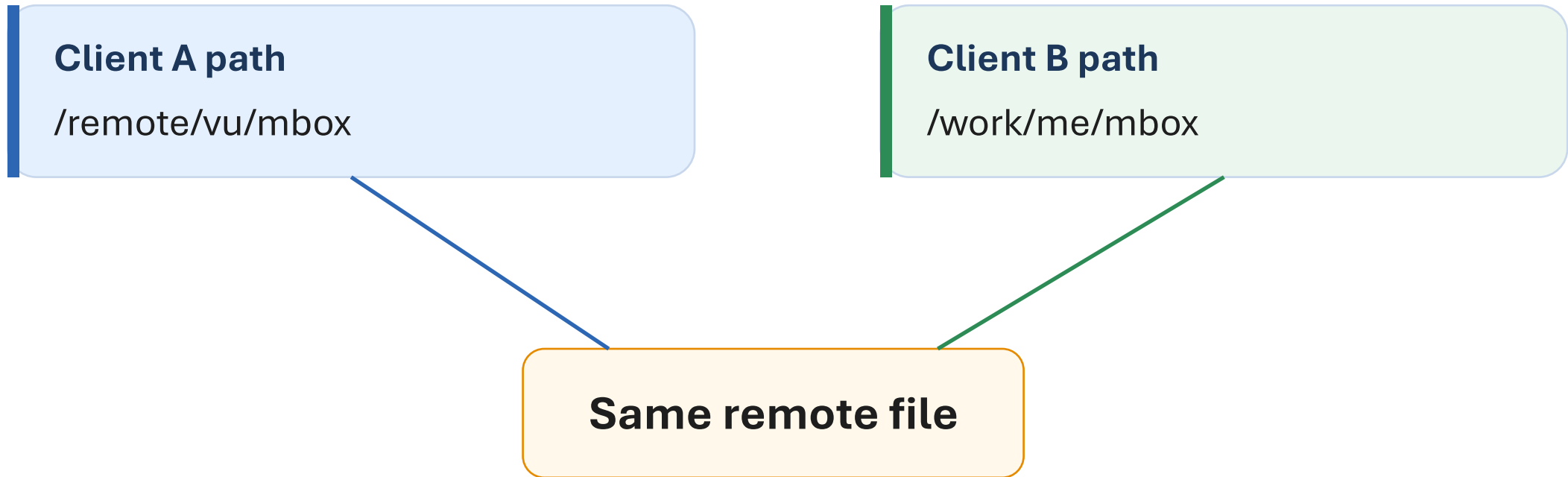
Remote server exports: `/users/alice`
Client mounts it as: `/home/alice`

User accesses: `/home/alice/report.txt`



命名问题：同一个文件，不同名称

Naming Problem: Same File, Different Names



This makes sharing harder.

A global or standardized name space can reduce confusion.

文件句

File Handles

- ❑ A file handle is an internal reference to a file.
- ❑ It is created by the server.
- ❑ It is opaque to the client.
- ❑ After LOOKUP, the client uses the handle for read/write.

1. Client gives file name
2. Server returns file handle
3. Client uses handle for READ / WRITE



自动安装 Automounting

- ❑ Automounting mounts a remote file system only when needed.
- ❑ It avoids mounting every possible directory at login.
- ❑ Useful for large systems with many users.

```
graph TD; A[User accesses /home/alice] --> B[Automounter finds server]; B --> C[Mounts Alice's home directory]; C --> D[Program continues];
```

Advantage: less startup overhead and fewer unnecessary mounts.

全球命名空间

Global Name Space

Goal: the same file should have the same name everywhere.

- ❑ Useful when many legacy file systems are combined.
- ❑ Makes sharing easier because users can refer to the same path.
- ❑ Location services can hide where the file is physically stored.

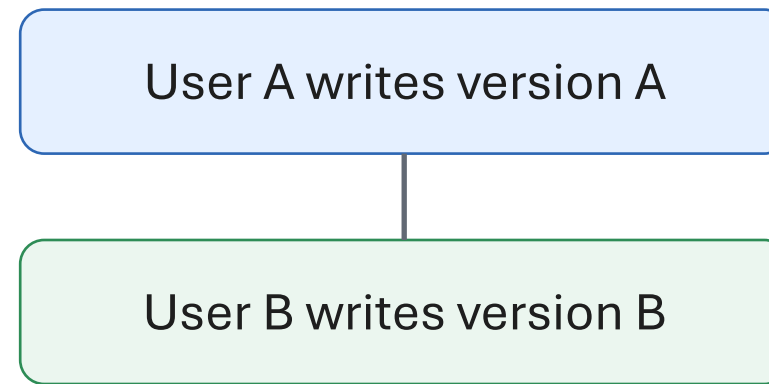
Without global naming:
Client A: /remote/vu/mbox
Client B: /work/me/mbox

With global naming:
Both use: /global/vu/mbox

文件共享问题

File Sharing Problem

- ❑ Two users may read and write the same file at the same time.
- ❑ The system must define what result is correct.
- ❑ This is called file-sharing semantics.



四种文件共享语义

Four File-Sharing Semantics

UNIX semantics

Every write is immediately visible to all processes.

Session semantics

Changes become visible when the file is closed.

Immutable files

Files cannot be updated; new versions are created.

Transactions

A group of operations happens atomically.

Main trade-off: strict correctness vs better distributed performance.

UNIX 语义与会话语义

UNIX Semantics vs Session Semantics

UNIX semantics

A writes “hello”. B immediately reads “hello”. Simple, but expensive with remote caches.

Session semantics

A writes locally. Other clients see changes only after A closes the file.

Session example:

1. A opens file
2. A writes changes
3. B may still read old version
4. A closes file
5. New version becomes visible

文件锁定

File Locking

- ❑ Locks prevent conflicting access.
- ❑ Read locks allow multiple readers.
- ❑ Write locks provide exclusive access.
- ❑ NFSv4 supports byte-range locks.

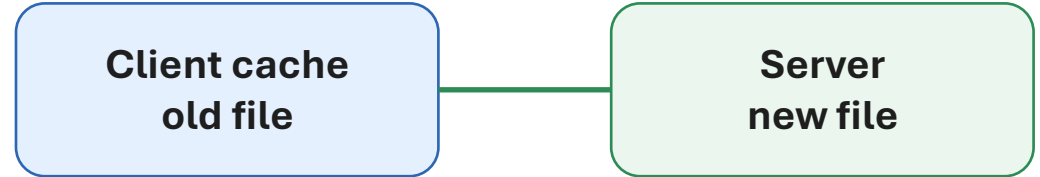
Client A
WRITE lock
bytes 0–100

Client B
cannot write
bytes 0–100

Lock operations:
LOCK → create lock
LOCKT → test conflict
LOCKU → unlock
RENEW → renew lease

客户端缓存

Client-Side Caching



- ❑ Caching keeps a local copy of file data.
- ❑ It improves performance and reduces network traffic.
- ❑ But cached data may become stale.

Problem: client may read an old cached copy.

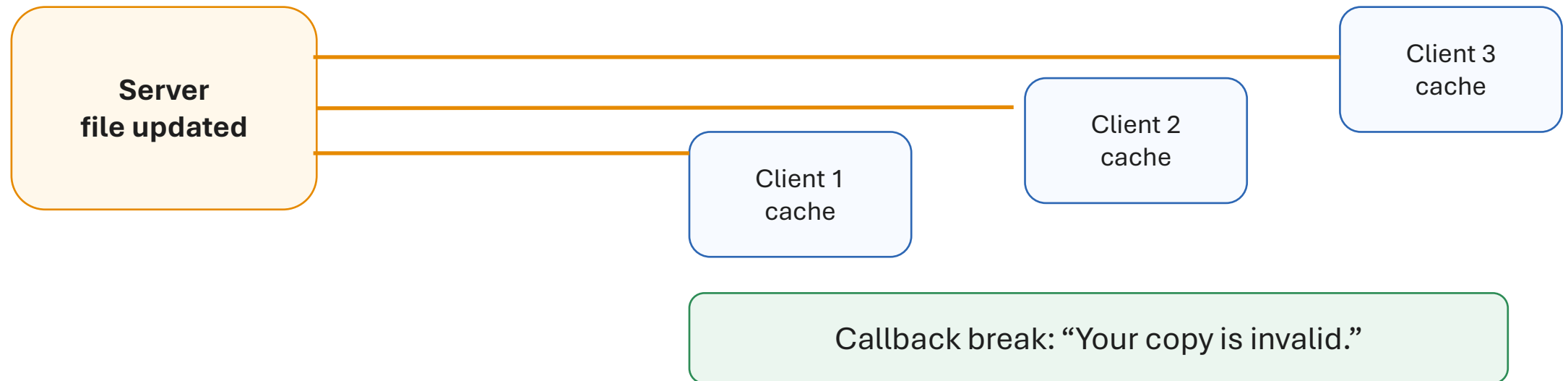
Cache revalidation:

1. Open cached file
2. Ask server for last modified time
3. If stale, discard cache
4. Otherwise, use cache

回调失效

Callback Invalidation

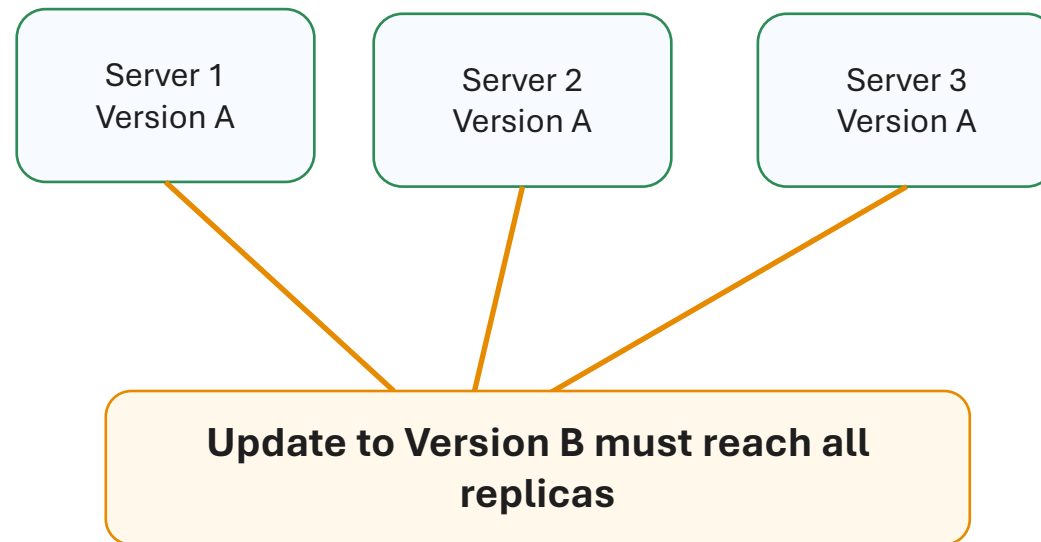
- ❑ Server remembers which clients cache a file.
- ❑ When the file changes, server sends invalidation.
- ❑ Client then discards or refreshes its stale copy.



服务器端复制

Server-Side Replication

- ❑ Stores copies of files or volumes on multiple servers.
- ❑ Improves availability and fault tolerance.
- ❑ Updates must be synchronized across replicas.



尾声：VSG和AVSG

Coda: VSG and AVSG

VSG: Volume Storage Group

All servers that store a copy of a volume.

AVSG: Accessible Volume Storage Group

Servers from the VSG that the client can currently contact.

$VSG = \{S1, S2, S3\}$

If S3 unreachable:
 $AVSG = \{S1, S2\}$

If AVSG is empty, the client is disconnected.

点对点复制

Version Vectors Detect Conflicts

- ❑ Version vectors record what each server has seen.
- ❑ They help detect independent updates during network partitions.
- ❑ Different vectors can reveal conflicts.

Initial on all servers: $[1, 1, 1]$

Client A updates S1 and S2:

$S1 = S2 = [2, 2, 1]$

Client B updates S3 separately:

$S3 = [1, 1, 2]$

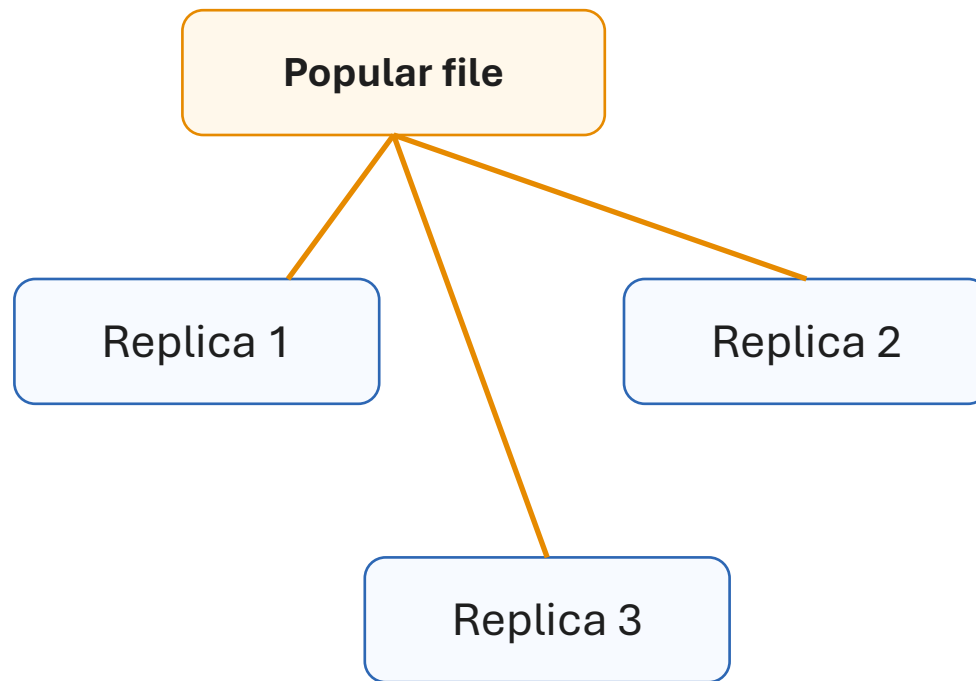
Conflict detected after reconnection.

Meaning: two groups edited the file independently.

点对点复制

Peer-to-Peer Replication

- ❑ Replication speeds up search.
- ❑ It improves availability.
- ❑ It helps balance load between nodes.
- ❑ Hard part: nodes frequently join and leave.



More replicas usually mean faster lookup and better availability.

容错性：什么会出错？

Fault Tolerance: What Can Fail?

Server crash

Client crash

Network partition

Disk failure

Node leaves system

Malicious server

A good distributed file system should keep data available and recover when possible.

Byzantine Fault Tolerance

- ❑ A Byzantine failure means a server may behave arbitrarily or maliciously.
- ❑ To tolerate k Byzantine failures, the system needs at least $3k + 1$ replicas.
- ❑ The client accepts a result confirmed by enough replicas.

Example:

$k = 1$ malicious/faulty server
Required servers = $3k + 1$
 = $3(1) + 1$
 = 4 servers

Basic phases: Request → Pre-prepare → Prepare → Commit → Reply

擦除编码

Erasure Coding

- ❑ 文件被拆分为 m 个片段。
- ❑ 它被编码为 n 个片段，其中 $n > m$ 。
- ❑ 任意 m 个编码的片段都可以重建该文件。
- ❑ 通常需要的存储空间比完整复制少。

Example:

$m = 4, n = 6$

Any 4 out of 6 fragments
can rebuild the original file.

- ❑ A file is split into m fragments.
- ❑ It is encoded into n fragments, where $n > m$.
- ❑ Any m encoded fragments can reconstruct the file.
- ❑ Usually needs less storage than full replication.



Need any 4 fragments

安全目标

Security Goals

Authentication

Who are you?

Authorization

What are you allowed to do?

Confidentiality

Can others read the data?

Integrity

Has data been changed?

Security protects file data, file metadata, and access operations.

NFS安全与ACLs

NFS Security and ACLs

- ❑ NFS uses secure RPC mechanisms for communication.
- ❑ NFSv4 supports RPCSEC_GSS.
- ❑ Access control is handled by file attributes such as ACLs.
- ❑ The server checks permissions before allowing operations.

Example ACL:

Alice: read, write

Bob: read only

Guest: no access

Authentication proves identity; authorization checks permissions.

安全文件系统与点对点安全

Secure File System and Peer-to-Peer Security

SFS idea

Self-certifying pathnames include information related to the server's public key, helping clients authenticate the server.

P2P security issues

Sybil attacks, eclipse attacks, malicious routing, fake data, and storage cheating.

- ❑ Centralized systems are easier to secure.
- ❑ Fully decentralized file systems need secure lookup and secure collaboration.

摘要

Summary

- ❑ Distributed file systems allow remote file sharing.
- ❑ NFS is a classic client-server distributed file system.
- ❑ Files can be accessed remotely or downloaded locally.
- ❑ Naming uses mounting and file handles.
- ❑ Caching improves performance but creates consistency problems.
- ❑ Replication improves availability but makes updates harder.
- ❑ Security needs authentication, authorization, and secure communication.

班级活动

Class Activity

Compare these real systems:

Google Drive

Dropbox

NFS

FTP

BitTorrent

Which one uses upload/download?

Which one uses remote access?

Which one uses replication?

Which one needs conflict handling?

Which one is peer-to-peer?

Goal: connect chapter ideas to systems students already know.